S-HPLB: Efficient LLM Attention Serving via Sparsity-Aware Head Parallelism Load Balance

Di Liu[1], **Yifei Liu**[1], **Chen Chen**[1,*], **Zhibin Yu**[2], **Xiaoyi Fan**[3], **Quan Chen**[1], **Minyi Guo**[1]
[1]Shanghai Jiao Tong University [2]Chinese Academy of Science [2]Shenzhen MSU-BIT University

Abstract

With the growing scale of Large Language Models (LLMs) and expanding context lengths, attention computation has become a key performance bottleneck in LLM serving. To accelerate attention computation, prior work often parallelizes attention heads across multiple GPUs. Each head performs sparse attention with a uniform sparsity budget, selectively computing only a subset of KV pairs. In this paper, we observe that attention heads within an LLM exhibit *heterogeneous-yet-stable* sparsity elasticities, which motivates us to enforce head-adaptive sparsity budgets to attain better efficiency while preserving high inference quality. Yet, from a systems perspective, heterogeneous sparsity levels lead to inconsistent computation times across heads, yielding cross-GPU resource bubbles under head-parallel deployment. To further minimize such bubbles, we propose a novel attention deployment strategy called *Sparsity-aware Head-Parallel Load Balance* (S-HPLB). Experiments on long-context benchmarks show that S-HPLB achieves a 2.88× reduction in decoding attention latency while maintaining output quality.

1 Introduction

Recent advances in large language models (LLMs) [1, 2, 3] have enabled remarkable capabilities in natural language understanding and generation. Meanwhile, applications such as document understanding [4], long-form question answering [5], and code synthesis [6, 7] feed models with increasingly longer contexts. As both model size and context length grow, the computational demand of LLM inference—especially the *attention* operation—escalates dramatically. For instance, when processing a 128K-token input with Llama-3.1-8B, the attention operation accounts for over 70% of the total time-to-first-token (TTFT) [8]. Reducing the cost of attention is therefore crucial for efficient LLM serving.

From the *system* perspective, recent work has adopted distributed attention deployment [9, 10, 11, 12], as illustrated in Figure 1. Specifically, to avoid cross-module resource interference, the attention module in each Transformer layer can be decoupled from the other modules (e.g., FFN), which is called attention-FFN disaggregation (AFD) [9, 10, 13]; this makes it possible to optimize attention efficiency independently. Furthermore, to speed up the attention module with expanded resources, leading LLM serving systems [11, 12] often distribute attention heads evenly across GPUs, a deployment mechanism we refer to as *Head-parallelism* (HP), which can be viewed as a special form of the Tensor Parallelism [14, 15].

In the meantime, from the *algorithm* perspective, *sparse attention* is a widely adopted optimization technique [16, 17, 18, 19]. Specifically, the attention operation exhibits inherent sparsity: each query vector primarily attends to only a subset of key vectors. By bypassing the attention computation of insignificant query-key pairs, sparse attention can effectively alleviate the computation cost. Moreover, in performing sparse attention, existing methods typically enforce a fixed token budget (i.e., adopting

*Corresponding author: Chen Chen.

Preprint.

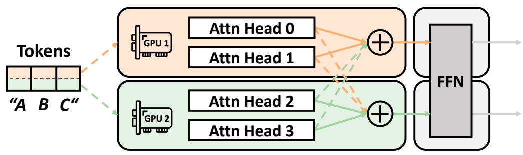

Figure 1: Illustration of distributed attention deployment. The attention module is decoupled from FFN via disaggregation, and heads are distributed evenly across GPUs.

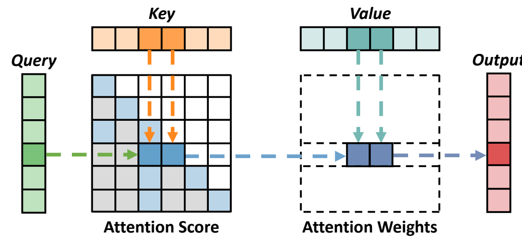

Figure 2: Illustration of the sparse attention workflow. For each query, a subset of key-value pairs is selected, and attention is computed only over the selected pairs.

the top-k policy where k is the selected token number) [17, 19], which, under HP, is indiscriminately applied to all the attention heads for balanced compute loads.

However, in terms of the efficiency-accuracy trade-off, we find that enforcing a uniform sparsity budget across different heads is not optimal. In fact, our testbed measurements (Figure 3) show that the attention heads within a Transformer layer consistently exhibit heterogeneous sparsity patterns (i.e., an identical sparsity budget leads to different cumulative attention scores on different heads). In that case, enforcing a uniform token budget would lead to redundant computations on some (high-sparsity) heads, yet at the same time incur large attention loss on other (low-sparsity) ones, failing to attain Pareto optimality in attention efficiency and inference accuracy.

Motivated by these observations, in this paper we devise a system-algorithm co-designed mechanism to improve the end-to-end attention efficiency while maintaining high accuracy. From the algorithm aspect, we first observe that, despite the cross-head sparsity heterogeneity, on each head the sparsity pattern is however stable across different use cases that have diverse input lengths and tasks; this makes it possible to conduct per-head sparsity modeling in an offline manner. Besides, given an expected sparsification ratio (i.e., the layer-level k budget) and the profiled per-head sparsity pattern, we propose a cross-head budget shifting strategy that reallocates budget from high-sparsity heads to low-sparsity ones, thereby improving accuracy without increasing the overall computation.

Nonetheless, when attention heads have different sparsity budgets, the attention computation time on different GPUs would be inconsistent, incurring resource wastage due to the existence of end-of-attention synchronization barriers. To further tackle that problem, from the system aspect, with the algorithm-side sparsity budgets, we model the head deployment problem as a classical multi-way partitioning problem, and propose a greedy heuristic to minimize the resource wastage caused by cross-GPU load inconsistency. We refer to our composite attention deployment scheme as *Sparsity-aware Head Parallelism Load Balance*, or S-HPLB.

We evaluate both the accuracy and efficiency of S-HPLB on three leading LLMs, running the widely adopted long-context benchmarks on a server equipped with eight A100 GPUs. Experimental results show that, compared to state-of-the-art sparse attention methods, S-HPLB improves model accuracy by up to 5.34% and reduces attention latency by up to 2.88×. Further deep-dive studies demonstrate that, S-HPLB operates consistently on the Pareto frontier of the latency-accuracy trade-off. Meanwhile, the head parallel load balancer alone contributes a 1.26× latency reduction. Our contributions are summarized as follows:

- We conduct a systematic measurement study on the sparsity characteristics of attention heads in LLMs, revealing that enforcing a uniform sparsity budget across heads is suboptimal (§2.2).
- We observe that cross-head sparsity heterogeneity is *stable* across diverse inputs, enabling reliable offline per-head sparsity profiling. We design a budget shifting algorithm that reallocates token budgets from high-sparsity heads to low-sparsity heads. We formulate the load balance problem as a multi-way partitioning problem and propose a greedy heuristic (§3).
- Experiments on three representative LLMs with the RULER benchmark show that S-HPLB achieves up to 5.34% accuracy improvement and 2.88× attention latency reduction compared to state-of-the-art sparse attention methods (§4).

2 Background and Motivation

2.1 Head Parallel Attention Deployment with Compute Sparsification

Attention: a key efficiency bottleneck in LLM inferences. A critical operation of LLM inference is self-attention [20]. Given an input of n tokens, self-attention first transforms them into three matrices: query Q, key K, and value V. Using the query and key, the attention weight A is derived and the output O is computed as the weighted sum over the value V:

$$A = \text{Softmax}(\frac{Q \times K^T}{\sqrt{d}}),\ O = A \times V \tag{1}$$

In mainstream attention architectures such as Multi-Head Attention (MHA) and Group-Query Attention (GQA) [21], the attention operation is typically composed of multiple modules, each known as an attention head (e.g., 32 heads for the Llama-3.1-8B model). Each head independently performs self-attention as in Eq. (1) and captures diverse features from different subspaces. The results from all heads are then aggregated to yield the output.

Moreover, *long context* LLM inferences, which support a wide range of applications like document understanding and repository-level code debugging [6, 7], are usually bottlenecked by the attention computation. Specifically, processing all input tokens at once incurs *quadratic* computational complexity [19, 22], making the attention operation easily *compute-bound*. As an illustration, the attention operation accounts for over 70% of the total Time-to-First-Token (TTFT) latency when handling a 128K-token input with the Llama-3.1-8B model[1]. Therefore, it is of paramount significance to optimize the attention efficiency for the popular long-context inferences.

Distributed attention deployment in *head-parallel* mode. Since attention has become a critical bottleneck, existing work has focused on optimizing attention serving through distributed model deployment [9, 10, 11, 12]. Given the differing computational characteristics between the attention and other operations, some studies have proposed *Attention-FFN Disaggregated* (AFD) deployment [9, 10]. AFD makes optimizing attention serving efficiency as the sole objective possible. For the attention operation itself, current serving systems primarily optimize inference latency through tensor parallelism. As discussed in Section 2.1, the multi-head nature of attention is inherently suitable for parallelization. Therefore, as shown in Figure 1, leading serving systems [11, 12] typically employ head parallelism (HP) for attention to improve serving quality.

Attention acceleration with sparse attention. Apart from the *system-side* distributed attention deployment, *sparse attention*, as an *algorithm-side* optimization technique, is also widely adopted for fast attention [17, 22, 19, 18, 26, 8, 23]. Sparse attention leverages the inherent sparsity of the attention mechanism to alleviate the quadratic complexity. As illustrated in Figure 2, this sparsity arises naturally from the weighted average in attention computation, where only the value vectors associated with high attention weights contribute most significantly to the attention output. For example, Jiang et al. [19] report that the top-4% of the total tokens account for 96.4% of the overall attention weight. Ideally, if a set of most-significant query-key pairs are selected in attention calculation, the inference can be substantially accelerated with negligible accuracy degradation.

For example, some works observe structured sparsity patterns of the attention map in prefill phase and accordingly optimize the quadratic computation overhead. StreamingLLM [27] proposes to only compute the initial and recent tokens in the context window, and MInference [19] combines different sparsity patterns (e.g., local window, vertical-slash and block sparse) when selecting the token candidates for attention computation. Despite with different selection principles, all those works rely on a critical hyperparameter—the token budget (k)—to specify the ultimate number of tokens that participate in attention computation on each attention head.

[1] In the decoding phase, key-value (KV) cache optimization reduces the complexity to linear, but the operation becomes memory-bound due to frequent memory accesses [18]. In this work, we focus on optimizing the computation cost in the prefill phase, complementing existing studies that optimize decoding attention [23, 24]. Besides, in some scenarios like prefill-only workloads [25], TTFT is even the only optimization target.

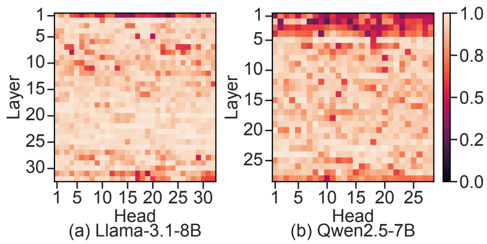

Figure 3: Recovery ratio of different attention heads under 4096 token budget.

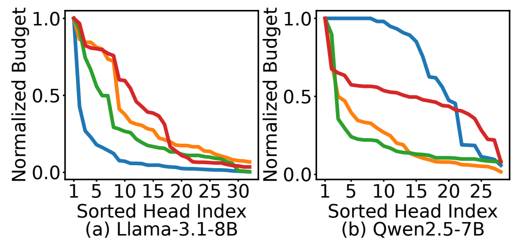

Figure 4: Per-head token budget required to achieve a 95% recovery ratio.

2.2 The Need for Sparse Attention with Head-Adaptive Token Budgets

While the above sparse-attention works enforce an identical token budget on each head, we find that different attention heads of an LLM often exhibit heterogeneous sparsity characteristics. That is, some attention heads require only a small number of tokens to recover high attention weight (i.e., A in Eq. (1)), while others do not. This heterogeneity leads to varying sensitivity across heads, rendering a uniform budget allocation inherently suboptimal.

To confirm, we quantify the attention sparsity by calculating the cumulative sum of attention weight of top-k critical tokens. This cumulative sum, called recovery ratio, indicates how much of the full attention can be recovered using a small number of critical tokens (a higher recovery ratio reflects a larger sparsity). We conduct a systematic analysis of per-head attention sparsity on Llama-3.1-8B [28] and Qwen2.5-7B [29] using the PG-19 dataset [30] with context length of 32K tokens. We profile the recovery ratio for the top-k (k=4096) tokens across different attention heads. As shown in Figure 3, there exists considerable heterogeneity in the recovery ratio of different attention heads. Such heterogeneity is also corroborated by some recent works [31, 32].

To further illustrate the potential of head-adaptive budgets, we fix the target recovery ratio at 95% and measure the per-head token budget required to reach this target. As shown in Figure 4, the required budgets vary significantly across heads. Notably, when all heads achieve a 95% recovery ratio, the average budget across heads is only 3,149 tokens for Llama-3.1-8B. In contrast, under a uniform budget of 4,096 tokens, the average recovery ratio is merely 89.97%. This result demonstrates that, by adaptively allocating budgets across heads, it is possible to achieve higher accuracy without increasing the overall computation cost.

3 S-HPLB Design

3.1 Architecture Overview

Our findings in Section 2.2 reveal the heterogeneity of attention sparsity across heads. This highlights the potential for adaptive per-head budget allocation to improve accuracy without sacrificing system efficiency. However, it is a non-trivial task to leverage such cross-head attention heterogeneity for efficient and accurate attention serving. On the one hand, we need to precisely determine the number of token candidates for sparsified attention computation on each head—so as to make substantial reduction on the overall computation amount. On the other hand, we need to minimize the resource wastage caused by inconsistent computation loads on different GPU devices under head parallelism.

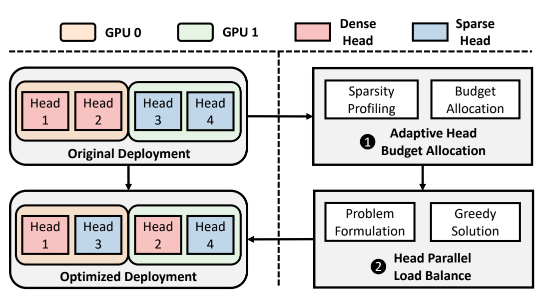

Figure 5: Architecture of S-HPLB.

In this work, we propose S-HPLB (Sparsity-Aware Head-Parallel Load Balancing), a novel system-algorithm co-designed framework for efficient LLM attention serving. S-HPLB is tailored to strike

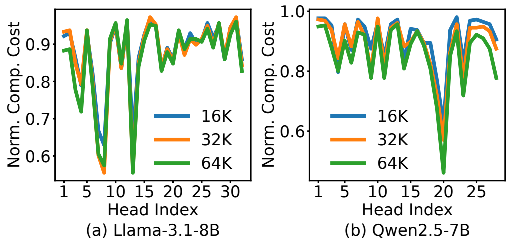

Figure 6: Normalized budget required by different heads to achieve a fixed attention weight recovery ratio under inputs of varying lengths.

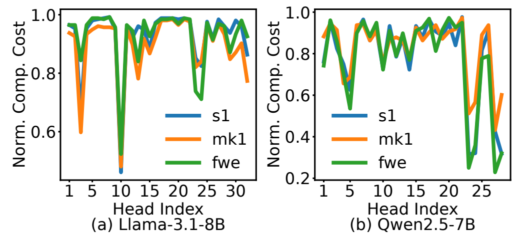

Figure 7: Normalized budget required by different heads to achieve a fixed attention weight recovery ratio under inputs of varying tasks.

an optimal balance between accuracy performance and computation efficiency. Figure 5 illustrates the system architecture of S-HPLB, which comprises two core components: adaptive head budget allocation and head parallel load balancing. First, S-HPLB profiles the heterogeneous sparsity patterns of individual attention heads across diverse inputs via offline profiling, and then adaptively shifts computational budgets across the heads using a max–min optimization strategy. Second, S-HPLB formulates the head-to-device assignment as a multi-way partitioning problem, and minimizes the inter-device load imbalance with a greedy heuristic.

3.2 Acquiring Per-head Sparsity Pattern

Given the heterogeneous sparsity across heads, a natural objective is to equalize the cumulative attention score (i.e., recovery ratio) across all heads, ensuring that each head retains a sufficient portion of its attention weight. This corresponds to a *top-p* strategy [22, 33], where each head dynamically selects the minimum number of tokens whose cumulative attention weight exceeds a threshold p. While top-p strategy can adapt to cross-head heterogeneity, it suffers from two significant drawbacks. First, unlike top-k methods that only require ordinal information to select the most important tokens, top-p demands precise online estimation of token importance to determine where the cumulative weight crosses the threshold, incurring substantial analysis overhead. Second, since the per-head budget is determined by the threshold rather than explicitly controlled, certain heads may retain a disproportionately large number of tokens to meet the threshold, leading to uncontrollable computation cost and increased latency.

Instead, we seek to approximate top-p behavior within the efficient top-k paradigm—assigning each head a predetermined token budget k_h such that the resulting recovery ratios are approximately equalized. This requires knowing the per-head sparsity curve (i.e., the mapping from token budget to recovery ratio) prior to inference. A key question then arises: how can we obtain these curves without runtime overhead?

Fortunately, we observe that the relative sparsity of individual attention heads exhibits remarkable stability across requests: although the absolute number of required tokens varies with context length, the proportion of tokens needed to achieve a given recovery ratio remains consistent across tasks and inputs. To validate this, we experiment with Llama-3.1-8B and Qwen2.5-7B on a series of calibration datasets [34], adjusting the per-head budget to reach a recovery ratio of 0.9 across various context lengths and task domains. As shown in Figure 6 and Figure 7, the per-head sparsity ratio remains highly stable across diverse input, consistent with findings in [31]. This stability enables *offline profiling*: a one-time calibration on a representative dataset suffices to obtain reliable per-head sparsity curves for runtime use.

3.3 Adaptive Head Budget Allocation

With the per-head sparsity curves obtained via offline profiling, we now address the budget allocation problem: given a total token budget K across all heads, how should we distribute it to approximate top-p accuracy within the top-k paradigm? We propose *max-min budget shifting*, a simple yet effective algorithm based on the principle of maximizing the minimum recovery ratio across all heads. The intuition is straightforward: heads with high sparsity can afford a smaller budget without significant accuracy loss, and the freed budget can be reallocated to low-sparsity heads that need it most.

As illustrated in Figure 8, the algorithm proceeds as follows. Initially, all heads are assigned an equal budget, resulting in heterogeneous recovery ratios (hollow circles). The algorithm then iteratively transfers budget from the head with the highest recovery ratio to the one with the lowest, thereby equalizing the recovery ratios across heads. The iteration terminates when one of the following conditions is met: (i) further reallocation yields no improvement, i.e., the budget-donating head becomes the new minimum (dashed line); or (ii) all heads have reached a predefined budget lower bound (e.g., 128 tokens).

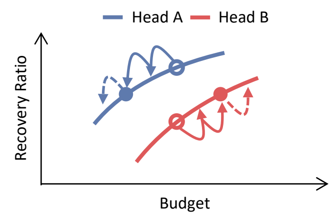

Figure 8: Illustration of the iterative max–min budget allocation process.

The max-min budget shifting offers two key advantages. First, by raising the minimum recovery ratio across heads, it prevents any single head from becoming an accuracy bottleneck—this is particularly important because overall model accuracy is often dominated by the worst-performing heads rather than the average. Second, the reallocation operates under a fixed total budget, meaning the accuracy improvement comes at no additional computation cost. In essence, max-min budget shifting achieves a Pareto improvement: better accuracy with the same efficiency.

3.4 Head Parallel Load Balance

While allocating different sparsity budgets to attention heads can better navigate between accuracy and efficiency, it however introduces cross-head load imbalance. Existing head-parallel (HP) methods [11, 12] assume that all attention heads have equal computational cost and thus assign them to GPUs indiscriminately; it is thus possible that the overall computing amounts on different GPUs are inconsistent. Figure 9 illustrates this imbalance when the 32 attention heads of a Llama-3.1-8B layer are sequentially distributed across 4 GPUs: the load imbalance reaches as high as 2.78×. Such cross-head computational inconsistency may lead to substantial GPU idleness, as faster GPUs are forced to wait for straggler devices to complete their computations. This highlights the need for a more effective load balance strategy to ensure consistent computation times across devices and improve overall resource utilization. In this subsection, we first formulate the load balancing optimization problem and subsequently present an efficient greedy algorithm, Head Parallel Load Balance, to resolve cross-GPU load inconsistency.

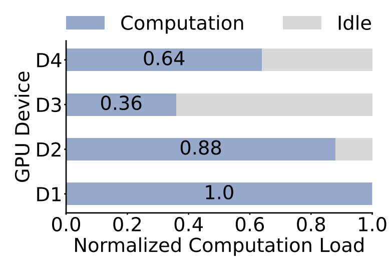

Figure 9: Load imbalance from naive head parallel attention deployment.

Problem formulation. To address the load imbalance issue, we formulate an optimization problem to efficiently allocate attention heads across multiple devices. Given a set of attention heads $H = \{h_1, h_2, \dots, h_N\}$ with corresponding budgets $b_{h_1}, b_{h_2}, \dots, b_{h_N}$. Let D represent the set of devices, and $H_d \in H$ be the subset of heads assigned to device $d \in D$. The optimization problem can be formulated as follows:

$$
\begin{aligned}
\min_{\{H_d\}_{d\in D}} \quad & \mathcal{I} = \frac{\max_{d\in D} L_d}{\frac{1}{|D|}\sum_{d\in D} L_d}, \\
\text{s.t.} \quad & L_d = \sum_{h\in H_d} b_h, \quad \forall d \in D, \\
& \bigcup_{d\in D} H_d = H, \\
& H_{d_1} \cap H_{d_2} = \emptyset, \quad \forall d_1 \neq d_2,
\end{aligned} \tag{2}
$$

where L_d denotes the total load of the device d. Our objective is to minimize the load imbalance ratio $\mathcal{I}$, subject to the constraint that each head is assigned to exactly one device.

A greedy heuristic for efficient problem solving. This attention head deployment problem is essentially an NP-hard multiway partitioning problem [35]. Although exact algorithms such as dynamic programming guarantee an optimal solution, their state space grows exponentially with the number of devices $|D|$ and the upper budget bound L, with a complexity of $O(N \cdot L^{|D|-1})$. This makes them computationally infeasible when either D or L is large. Therefore, we employ an efficient greedy algorithm to solve this problem. The algorithm begins by sorting attention heads in descending order of their budgets. Each attention head is then assigned to the device with the currently smallest load. This process continues until all heads have been allocated. The greedy approach minimizes the load imbalance by prioritizing the allocation of the most demanding heads first, ensuring a balanced load distribution. This algorithm has a relatively low time complexity of $O(N \log N + N \log K)$, making it suitable for adoption in practice.

4 Evaluation

In this section, we comprehensively evaluate S-HPLB against full attention and state-of-the-art sparse attention methods. We present accuracy and efficiency results in Section 4.2, microscopic studies in Section 4.3, and ablation studies in Section 4.4.

4.1 Experimental Setup

Model, benchmark and hardware testbed. We evaluate S-HPLB primarily on three leading open-source LLMs: Llama-3.1-8B [28], Qwen2.5-7B [36], and Qwen2.5-72B [29]. All models support a context window of up to 128K tokens. Moreover, we include the Llama-3-8B-262K model [37] in our evaluation to assess efficiency and scalability in scenarios with longer contexts. We evaluate model accuracy on two widely-used long-context benchmark. (1) ∞-Bench [38]: this benchmark consists of retrieval tasks and realistic tasks. The average context length of ∞-Bench is over 100K tokens. (2) RULER [34]: this benchmark consists of four categories and thirteen tasks, including retrieval, multi-hop tracing, aggregation, and QA, with context lengths of up to 128K. All experiments were conducted on a single server equipped with an Intel Xeon Platinum 8369B CPU and eight NVIDIA A100 80GB GPUs connected via NVLink.

Baselines. We compare S-HPLB against several strong baselines representing the state-of-the-art in efficient LLM inference. For full attention baseline we run FlashAttention [39]. For sparse attention, we select representative methods from two categories: (1) Top-k methods: StreamingLLM [17] and MInference [19]. These methods rely on a predefined computational budget, but differ in how the budget is allocated: StreamingLLM adopts an initial and tail local window, while MInference employs three distinct sparse attention patterns. (2) Top-p method: XAttention [22]. This method dynamically determines the budget at runtime to ensure the cumulative attention weight meets a predefined threshold. We set the hyperparameter for all methods according to original papers.

Table 1: Accuracy (%) comparison of different models and methods on RULER. S-HPLB outperforms all other methods and maintains accuracy comparable to that of full attention across all models.

	Methods	NS1	NS2	NS3	MK1	MK2	MK3	MV	MQ	VT	CWE	FWE	QA1	QA2	**Avg.**
Llama-3.1-8B	Full Attention	100.00	100.00	99.00	99.00	87.00	58.00	91.50	97.75	90.00	0.00	51.67	76.00	43.00	76.38
	StreamingLLM	93.00	48.00	38.00	32.00	6.00	0.00	35.75	33.75	68.60	0.10	58.67	16.00	24.00	34.91
	MInference	99.00	97.00	100.00	95.00	78.00	33.00	84.25	94.00	79.00	0.20	52.33	65.00	40.00	70.52
	XAttention	100.00	100.00	99.00	97.00	84.00	44.00	92.50	97.50	89.20	0.20	49.33	56.00	44.00	73.29
	S-HPLB	100.00	99.00	100.00	95.00	86.00	63.00	86.75	96.50	86.80	0.50	58.67	72.00	42.00	**75.86**
Qwen2.5-7B	Full Attention	100.00	100.00	92.00	93.00	49.00	17.00	69.50	86.25	83.80	37.70	62.76	50.00	36.00	67.46
	StreamingLLM	77.00	84.00	88.00	27.00	4.00	1.00	45.25	34.25	23.40	10.80	73.00	11.00	21.00	38.44
	MInference	100.00	96.00	96.00	90.00	47.00	5.00	59.00	81.50	72.60	20.90	66.00	36.00	33.00	61.77
	XAttention	100.00	100.00	95.00	84.00	22.00	13.00	69.00	90.75	75.60	37.50	64.00	35.00	39.00	63.45
	S-HPLB	100.00	100.00	96.00	94.00	59.00	11.00	67.00	90.50	70.60	30.10	63.00	38.00	40.00	**66.09**
Qwen2.5-72B	Full Attention	100.00	100.00	98.00	100.00	82.00	28.00	96.50	100.00	100.00	92.80	86.67	54.00	50.00	83.69
	StreamingLLM	10.00	26.00	92.00	26.00	6.00	0.00	9.50	48.50	4.40	3.00	87.50	18.00	24.00	27.30
	MInference	100.00	100.00	100.00	92.00	74.00	38.00	90.50	99.50	85.20	74.00	86.81	42.00	54.00	79.69
	XAttention	100.00	100.00	98.00	96.00	58.00	20.00	93.00	98.00	96.00	88.80	87.50	46.00	58.00	79.95
	S-HPLB	100.00	100.00	100.00	98.00	80.00	48.00	92.00	100.00	82.80	79.00	85.42	34.00	48.00	**80.56**

Table 2: Accuracy (%) comparison of different models and methods on ∞-Bench.

	Methods	En.Sum	En.QA	En.MC	En.Dia	Zh.QA	Code.D	Math.F	Retr.P	Retr.N	Retr.K	Avg.
Llama-3.1-8B	Full Attention	29.14	9.15	64.00	12.00	17.09	0.00	86.00	100.00	98.00	26.00	44.14
	StreamingLLM	27.32	5.69	38.00	16.00	11.48	8.00	90.00	96.00	76.00	0.00	36.85
	MInference	28.60	8.19	62.00	10.00	14.46	2.00	96.00	100.00	94.00	16.00	43.13
	XAttention	27.53	8.70	62.00	20.00	17.85	0.00	84.00	100.00	98.00	34.00	45.21
	S-HPLB	27.10	8.68	58.00	20.00	15.73	2.00	86.00	100.00	94.00	32.00	44.35
Qwen2.5-7B	Full Attention	30.17	7.51	70.00	14.00	10.90	24.00	100.00	100.00	100.00	8.00	46.46
	StreamingLLM	31.30	5.53	40.00	6.00	8.41	26.00	100.00	72.00	20.00	0.00	30.92
	MInference	27.26	6.07	60.00	8.00	9.93	26.00	100.00	100.00	100.00	0.00	43.73
	XAttention	29.57	6.93	64.00	12.00	12.04	22.00	100.00	100.00	98.00	6.00	45.05
	S-HPLB	27.46	5.72	60.00	8.00	10.61	30.00	100.00	100.00	100.00	0.00	44.18

4.2 End-to-end Results

Accuracy evaluation. Table 1 reports the accuracy comparison on the RULER benchmark. As a top-k method with adaptive budget allocation, S-HPLB consistently outperforms the top-k baseline MInference by 5.34%/4.32%/0.87% on Llama-3.1-8B/Qwen2.5-7B/Qwen2.5-72B, respectively. The improvement is particularly pronounced on challenging multi-key retrieval tasks (MK2, MK3), where uniform budgets tend to under-allocate for retrieval-heavy heads. Meanwhile, S-HPLB even achieves accuracy improvement comparing with the top-p method XAttention, with a gap of 2.57%/2.64%/0.61% on the three models. This indicates the effectiveness of adaptive budget shifting algorithm. Intuitively, the top-p method should achieve the best accuracy. However, in practice, due to the inaccurate and computationally expensive estimation of attention weights, somtimes its performance is suboptimal. Note that for certain tasks (e.g., MK3 on Llama3.1-8B, and also for some points in later Figure 13), S-HPLB achieves even higher accuracy than full attention. This improvement can be attributed to S-HPLB's ability to filter out noise and help the model concentrate on critical tokens. Other sparse attention studies have also reported this phenomenon [24].

Table 2 presents results on the ∞-Bench, where a similar trend holds: S-HPLB consistently outperforms MInference and achieves comparable accuracy to XAttention across both models. This is consistent with our design goal: approximating top-p accuracy within the more efficient top-k paradigm. These results confirm that head-adaptive budget allocation effectively bridges the accuracy gap between top-k and top-p methods, which, as we show next, also brings significant efficiency advantages.

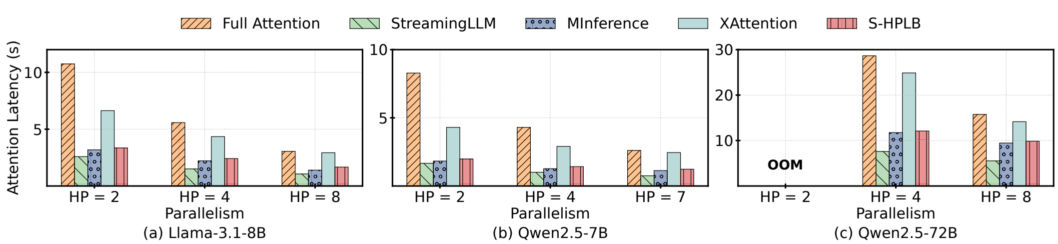

Figure 10: Latency of attention serving across models with 128K input. S-HPLB outperforms the top-p method and achieves comparable performance to the top-k method.

Efficiency evaluation. We evaluate the attention serving latency across different models and degrees of parallelism under a 128K context length. As shown in Figure 10, S-HPLB achieves $3.39\times$/$4.27\times$/$3.31\times$ lower latency than full attention on the three evaluated models. In comparison with sparse attention baselines, S-HPLB reduces latency by $2.09\times$/$2.22\times$/$2.88\times$ relative to XAttention. Benefited from head-parallel load balancing, S-HPLB also attains latency on par with top-k methods MInference.

4.3 Microscopic Study

Figure 11 reports the average bubble ratio across different numbers of GPUs for three models. The naive sequential partition suffers from increasingly severe imbalance as parallelism scales. S-HPLB

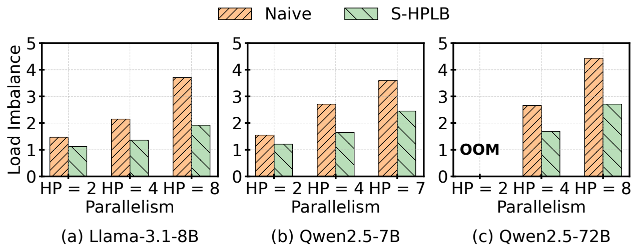

Figure 11: Comparison of load imbalance between sequential placement and S-HPLB. Our method consistently outperforms sequential placement.

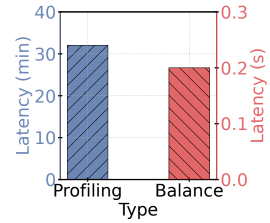

Figure 12: Comparison of latency between budget shift profiling and load balance phases.

consistently reduces the bubble ratio, achieving up to 48% reduction on Llama-3.1-8B and 39% on Qwen2.5-72B. These results confirm that sparsity-aware assignment effectively mitigates the load imbalance inherent in head parallelism.

As shown in Figure 12, the overhead of S-HPLB is negligible. The budget-shifting profiling is a one-time offline cost of approximately 32 minutes, and the subsequent load balancing assignment completes in under 0.2 seconds. Both steps are performed only once per model and can be reused across all deployment configurations.

We further analyze how varying the configurations impacts overall performance. For top-k-based methods, we vary the uniform budget k, while for top-p-based methods, we adjust the threshold p. For S-HPLB, we modify the total budget across all heads. These settings directly control the sparsity level. We evaluate the performance of all methods on the challenging MK2 task from the RULER benchmark [34]. Figure 13 shows the curves between latency (normalized by the latency of full attention) and accuracy for various methods. S-HPLB consistently occupies a more favorable position on the Pareto frontier, offering a better trade-off between accuracy and efficiency.

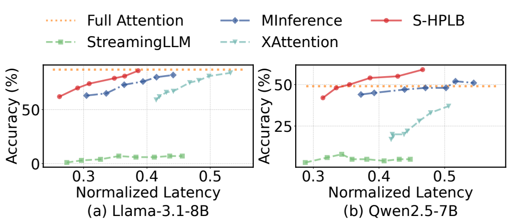

Figure 13: Accuracy-latency trade-off (Skyline) across various methods on two models.

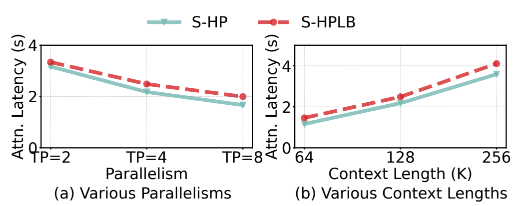

Figure 14: Impact of load balance under different parallelism degrees and context lengths.

4.4 Ablation Study

To check the effectiveness of our head parallel load balancer, we experiment respectively with and without the load balancer. We measure the attention latency across different degrees of parallelism with a fixed context length of 128K using the Llama-3.1-8B model. Additionally, we evaluate latency across different context lengths with a parallelism degree of 4. As shown in Figure 14, the load balancer consistently reduces latency across all configurations. Across different parallelism degrees and context lengths, it achieves a latency reduction of up to $1.19\times$ and $1.26\times$, respectively.

5 Conclusion

This paper presents S-HPLB, a system-algorithm co-designed mechanism to improve the attention serving efficiency while maintaining accuracy. S-HPLB leverages the sparsity stability of attention heads and introduces a novel adaptive head budget allocation algorithm. It further mitigates tail latency stemming from cross-GPU load inconsistency by employing head parallel load balance. Experimental results show that S-HPLB achieves an optimal trade-off between latency and accuracy, consistently operating on the Pareto frontier compared to the state-of-the-art sparse attention baselines.

References

[1] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[2] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[3] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[4] Minzheng Wang, Longze Chen, Fu Cheng, Shengyi Liao, Xinghua Zhang, Bingli Wu, Haiyang Yu, Nan Xu, Lei Zhang, Run Luo, Yunshui Li, Min Yang, Fei Huang, and Yongbin Li. Leave no document behind: Benchmarking long-context llms with extended multi-doc QA. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, November 12-16, 2024*, 2024.

[5] Tianle Li, Ge Zhang, Quy Duc Do, Xiang Yue, and Wenhu Chen. Long-context llms struggle with long in-context learning. *Trans. Mach. Learn. Res.*, 2025.

[6] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations, ICLR 2024*, 2024.

[7] Ramakrishna Bairi, Atharv Sonwane, Aditya Kanade, Vageesh D. C., Arun Iyer, Suresh Parthasarathy, Sriram K. Rajamani, Balasubramanyan Ashok, and Shashank Shet. Codeplan: Repository-level coding using llms and planning. *Proc. ACM Softw. Eng.*, 2024.

[8] Di Liu, Meng Chen, Baotong Lu, Huiqiang Jiang, Zhenhua Han, Qianxi Zhang, Qi Chen, Chengruidong Zhang, Bailu Ding, Kai Zhang, Chen Chen, Fan Yang, Yuqing Yang, and Lili Qiu. Retrievalattention: Accelerating long-context LLM inference via vector retrieval. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.

[9] Ruidong Zhu, Ziheng Jiang, Chao Jin, Peng Wu, Cesar A Stuardo, Dongyang Wang, Xinlei Zhang, Huaping Zhou, Haoran Wei, Yang Cheng, et al. Megascale-infer: Efficient mixture-of-experts model serving with disaggregated expert parallelism. In *Proceedings of the ACM SIGCOMM 2025 Conference*, 2025.

[10] Bin Wang, Bojun Wang, Changyi Wan, Guanzhe Huang, Hanpeng Hu, Haonan Jia, and Nie Hao. Step-3 is large yet affordable: Model-system co-design for cost-effective decoding. *CoRR*, 2025.

[11] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark W. Barrett, and Ying Sheng. Sglang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024.

[12] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, 2023.

[13] XDS Team. xdeepserve: Model-as-a-service on huawei cloudmatrix384, 2025.

[14] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

[15] Zizhao Mo, Jianxiong Liao, Huanle Xu, Zhi Zhou, and Chengzhong Xu. Hetis: Serving llms in heterogeneous gpu clusters with fine-grained and dynamic parallelism. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1710–1724, 2025.

[16] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark W. Barrett, Zhangyang Wang, and Beidi Chen. H2O: heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023.

[17] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning Representations, ICLR 2024*, 2024.

[18] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context llm inference. In *Forty-first International Conference on Machine Learning, ICML 2024*, 2024.

[19] Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H. Abdi, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Minference 1.0: Accelerating pre-filling for long-context llms via dynamic sparse attention. In *Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024*, 2024.

[20] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Annual Conference on Neural Information Processing Systems 2017*, 2017.

[21] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*, 2023.

[22] Ruyi Xu, Guangxuan Xiao, Haofeng Huang, Junxian Guo, and Song Han. Xattention: Block sparse attention with antidiagonal scoring. *arXiv preprint arXiv:2503.16428*, 2025.

[23] Zhuoming Chen, Ranajoy Sadhukhan, Zihao Ye, Yang Zhou, Jianyu Zhang, Niklas Nolte, Yuandong Tian, Matthijs Douze, Léon Bottou, Zhihao Jia, and Beidi Chen. Magicpig: Lsh sampling for efficient llm generation. In *The Thirteenth International Conference on Learning Representations, ICLR 2025*, 2025.

[24] Yaoqi Chen, Jinkai Zhang, Baotong Lu, Qianxi Zhang, Chengruidong Zhang, Jingjia Luo, Di Liu, Huiqiang Jiang, Qi Chen, Jing Liu, Bailu Ding, Xiao Yan, Jiawei Jiang, Chen Chen, Mingxing Zhang, Yuqing Yang, Fan Yang, and Mao Yang. Retroinfer: A vector-storage approach for scalable long-context llm inference. *arXiv preprint arXiv:2505.02922*, 2025.

[25] Kuntai Du, Bowen Wang, Chen Zhang, Yiming Cheng, Qing Lan, Hejian Sang, Yihua Cheng, Jiayi Yao, Xiaoxuan Liu, Yifan Qiao, et al. Prefillonly: An inference engine for prefill-only workloads in large language model applications. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, pages 399–414, 2025.

[26] Hailin Zhang, Xiaodong Ji, Yilin Chen, Fangcheng Fu, Xupeng Miao, Xiaonan Nie, Weipeng Chen, and Bin Cui. Pqcache: Product quantization-based kvcache for long context LLM inference. *Proc. ACM Manag. Data*, 2025.

[27] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning Representations*, 2024.

[28] Meta. Llama-3.1-8b-instruct. `https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct`, 2024.

[29] Meta. Qwen2.5-72b-instruct. `https://huggingface.co/Qwen/Qwen2.5-72B-Instruct`, 2024.

[30] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, Chloe Hillier, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2020.

[31] Wenhao Wu, Yizhong Wang, Guangxuan Xiao, Hao Peng, and Yao Fu. Retrieval head mechanistically explains long-context factuality. In *The Thirteenth International Conference on Learning Representations, ICLR 2025*, 2025.

[32] Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, junxian guo, Shang Yang, Haotian Tang, Yao Fu, and Song Han. Duoattention: Efficient long-context LLM inference with retrieval and streaming heads. In *The Thirteenth International Conference on Learning Representations*, 2025.

[33] Chaofan Lin, Jiaming Tang, Shuo Yang, Hanshuo Wang, Tian Tang, Boyu Tian, Ion Stoica, Song Han, and Mingyu Gao. Twilight: Adaptive attention sparsity with hierarchical top-p pruning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.

[34] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, and Boris Ginsburg. RULER: What's the real context size of your long-context language models? In *First Conference on Language Modeling*, 2024.

[35] Jason Cong and Sung Kyu Lim. Multiway partitioning with pairwise movement. In *Proceedings of the 1998 IEEE/ACM international conference on Computer-aided design*, 1998.

[36] Qwen. Qwen2.5-7b-instruct. `https://huggingface.co/Qwen/Qwen2.5-7B-Instruct`, 2024.

[37] gradient. Llama-3-8b-instruct-262k. `https://huggingface.co/gradientai/Llama-3-8B-Instruct-262k`, 2024.

[38] Xinrong Zhang, Yingfa Chen, Shengding Hu, Zihang Xu, Junhao Chen, Moo Hao, Xu Han, Zhen Thai, Shuo Wang, Zhiyuan Liu, and Maosong Sun. ∞Bench: Extending long context evaluation beyond 100K tokens. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15262–15277, Bangkok, Thailand, 2024. Association for Computational Linguistics.

[39] Tri Dao, Daniel Y Fu, Stefano Ermon, Atri Rudra, and Christopher Re. Flashattention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.